31927- Applications Development with .NET

32998- .NET Applications Development

SPRING 2026

ASSIGNMENT-2 COVER SHEET

PROJECT TITLE	Personal Workout Tracker
LEAD STUDENT NAME	Daniel Mercuri
LEAD STUDENT ID	25320906
GROUP MEMBER NAME(S) AND STUDENT ID(S)	14393464 Ily Ralitza Perez 26147569 Vishvak Ananthakrishnan Rameshkumar
SUBMISSION DATE	17/19/2026

AI USE RESTRICTIONS FOR PROGRAMMING WORK

Do not use AI to generate a substantial or complete solution to the programming assignment, or to bypass the learning outcomes of the assessment.

- AI must not be used to invent or misrepresent code execution, test results, screenshots, datasets, performance measurements, external libraries/APIs used, contributions of group members, or any other project detail.
- Permitted support may include concept explanations, syntax help, debugging suggestions, code review, test-case ideas, partial code generation, documentation/comments, and limited code examples where consistent with the assessment instructions.
- You must understand, review, run and test any AI-assisted code yourself, verify that it meets the specification, and be able to explain every submitted component and design decision.
- All AI use must be disclosed. AI output is not an authoritative source and does not replace reading subject materials, documentation, or instructions from teaching staff.

DECLARATION OF ORIGINALITY AND RESPONSIBILITY

☐ I confirm that the submitted work is the work of the named student(s), except where collaboration, source code, libraries, data, or AI assistance has been appropriately acknowledged. I understand the submitted program and can explain its implementation, testing and results.

Lead student signature: D.Mercuri Date: 19/10/2026

Group member signature(s):
Ily Ralitza Perez Date: 19/10/2026
Vishvak Ananthakrishnan Rameshkumar

AI USE STATEMENT — ASSIGNMENT-2

Attach this completed statement as an appendix to your submission. Disclose all AI-enabled tools used, including uses that did not produce code retained in the final submission.

Student name(s) and ID(s)	25320906 Daniel Mercuri, 14393464 Ily Ralitza Perez, 26147569 Vishvak Ananthakrishnan Rameshkumar
Subject code	☐ 31927 ☐ 32998
Project / assessment title	Personal Workout Tracker

☐ I did use any AI-enabled tool for this assessment. (If selected, proceed to the declaration.)

1. AI tools and model names

List every AI-enabled tool used (for example ChatGPT, GitHub Copilot, Microsoft Copilot, Gemini, Claude, Grammarly or other coding assistants). Record the model name shown in the tool, or "not shown".

AI tool or product	Model name (as shown)	Date(s) used

2. What the AI tools were used for (tick all that apply)

☐ Explained programming concepts or APIs	☐ Suggested algorithms, approaches or pseudocode
☐ Helped with syntax or small code examples	☐ Generated or completed code
☐ Debugged errors or explained error messages	☐ Reviewed/refactored code I had written
☐ Suggested unit tests, edge cases or test data	☐ Generated documentation, comments or README text
☐ Helped with data analysis or visualisation code	☐ Assisted with version-control / build / tooling commands
☐ Proofread or revised report text	☐ Other: _____

3. Representative examples of AI use

Give 2–3 examples. Summarise what you asked the tool to do and explain what you accepted, changed, tested or rejected. Do not paste full chat logs unless requested.

Tool / model	Prompt or task summary	How I used, changed or rejected the output	Where it affected the submission

4. Verification and validation

☐ Ran and tested all submitted code and reviewed outputs, errors and edge cases	☐ Checked AI-suggested APIs/libraries against official documentation or course materials

☐ Confirmed that reported outputs, screenshots and performance results came from my/our actual program execution	☐ Reviewed AI-assisted code for correctness, security, originality, licensing/attribution and assessment requirements
☐ I/we can explain every submitted code component and justify the implementation and testing decisions	☐ No confidential, personal or unauthorised third-party material was uploaded to an AI tool

Briefly describe one specific check you performed and any change made as a result:

_____

_____

Declaration

I/we confirm that this statement accurately describes all AI-enabled tools used for this assessment and the extent to which their outputs influenced or appear in the submission. I/we followed the assessment instructions, independently reviewed and tested the submitted work, and take responsibility for the final code, report and results. I/we understand that prompts, outputs, commit history or other supporting evidence may be requested for an academic integrity review.

Lead student signature: D.Mercuri Date: 19/10/2026

Group member signature(s): Ily Ralitza Perez, Vishvak Ananthakrishnan Rameshkumar Date: 19/10/2026

Assignment 3
Personal Workout Tracker

Authors:

25320906 Daniel Mercuri

14393464 Ily Ralitza Perez

26147569 Vishvak Ananthakrishnan Rameshkumar

Introduction

Project Description

The personal workout tracker is a windows forms application developed using C# and .NET to help users organise their workout routines and track their exercise performance. The application includes several features such as the Dashboard, Workout Builder, Active Workout and progress Tracker.

The Dashboard acts as the main screen where users can view their selected workout and navigate between different features. The Workout Builder allows users to choose between Arm Day, Leg Day and Full Body Day. Users can add, edit, remove and save exercises based on their preferences. They can also customise exercise details such as the number of sets, repetitions, weight and duration.

The Active Workout feature allows users to record their exercise performance while completing a workout. It also includes a timer for timed exercises and a progress bar to show workout completion.

The Progress Tracker allows users to review their completed workouts through statistics and graphs. It displays information such as maximum weight lifted, repetitions, total training volume, personal records and workout frequency.

The application is designed for a single user, so a login system is not included.

Purpose

The main purpose of the Personal Workout Tracker is to provide users with a simple and organised way to manage their workout routines and monitor their fitness progress.

Instead of manually writing down exercises or remembering workout details, users can manage their routines in one application. The Workout Builder allows users to customise their workouts according to their preferences, while the Active Workout feature helps them record their performance during exercise sessions.

Another important purpose is to help users understand their progress over time. The Progress Tracker provides statistics and graphs that allow users to compare their workout results and identify improvements in areas such as strength, repetitions and workout consistency.

Overall, the application aims to make workout planning, exercise tracking and progress monitoring more convenient and easier to manage.

Requirements and Scope

Requirement	Implementation
At least one example of polymorphism which achieves a useful purpose (either through inheritance, method/constructor overloading/overriding	Exercise subclasses
At least two examples of Interface	INavigatable.cs IProgressBar.cs
At least one example of NUnit tests	UnitTest1.cs — NUnit test for the SaveWorkoutExercises() method in WorkoutService.cs
At least one example of Anonymous method with LINQ using Lambda expression	Queries in ProgressTracker.cs
At least one example of Generics/Generic based Collection	List<Workout> in WorkoutService.cs
At least four GUI forms/screens/interfaces which are resizable & responsive that have their own distinct feature or functionality	Dashboard.cs ActiveWorkout.cs ProgressTracker.cs ExerciseSelector.cs
At least six different unique categories of UI elements have been used	Label, Text Box, Button, Progress Bar, Data Grid View, Panel, ComboBox, ListBox
Use of entity framework	EF Core in DbContext.cs

Development approach

Model Structure

This project was created with a code-first approach. Development started with defining the key domain classes. The Exercise model was established first alongside its subclasses StrengthExercise and TimedExercise. They define the planned exercise through fields such as targeted sets or duration.

The Workout model was created which has a list of exercises and a name such as ‘Arm Day’ or ‘Leg Day’, functioning as a planned workout.

The ExerciseLog model was created which is the actual entry of an exercise and records the user’s achieved reps, sets, duration etc as well as the status and date of completion which allows for the tracking of the user’s workout history and progress tracking.

Finally, the WorkoutLog was created to group the individual exercise logs under their corresponding workout. It stores the recorded workout in its respective group such as ‘Arm Day’ as well as the completion status and date.

There were also several helper classes created. The first being WorkoutService which was used to save and edit workouts and logs. The second being FormControls which defined common form actions which reduces duplicated code. The last was DashboardRow in order to easily write up a planned exercise on the dashboard, regardless of its subclass.

Language Integrated Query (LINQ)

LINQ was used to query the SQLite Database through Entity Framework Core and to analyse workout history in memory. In WorkoutService, Include() and AsNoTracking() load saved workouts with their exercises, and IsWorkOutCompleted() queries the logs for the selected date. The Progress Tracker applies Where(), Select(), SelectMany(), GroupBy(), OrderBy(), Max(), Average(), Sum() to the completed workout logs using lambda expressions, for example flattening each workout's exercise logs with SelectMany() and totalling training volume with Sum(). Using LINQ for both kept data access consistent and avoided handwritten SQL and manual loops.

Personal Record Methods

UpdatePersonalRecords() in ProgressTracker.cs groups every completed ExerciseLog by exercise name and selects each group's best entry by highest weight, then highest reps. The ten most recent records are shown in the Recent personal records list. UpdateStatistics() uses the same ordering to find the best set for the selected exercise. Because records are calculated from the stored logs, they persist across restarts.

Entity Framework Core

SQLite persistence and Database configuration

WorkoutDbContext defines DbSet<Workout>, DbSet<Exercise>, DbSet<WorkoutLog> and DbSet<ExerciseLog>. OnConfiguring stores workouts.db in the user's LocalApplicationData/PersonalWorkoutTracker folder. EnsureCreated() does not migrate an existing schema, so the foreign key in the exisiting database was updated to match the final configuration.

Entity Relationships and Records

Each entity has a primary key. StrengthExercise, TimedExercise and CardioExercise share one Exercises table through a discriminator. A Workout has many Exercises with DeleteBehaviorSetNull, so removing an exercise from a plan keeps historical records. A WorkoutLog refers to its Workout (Restrict) and ows its ExerciseLogs (Cascade), and each ExerciseLog refers to its Exercise (Restrict)

Workout Builder and Logging Workflow

WorkoutService loads workouts with Include() and AsNoTracking() and seeds the default workout plans on first run. SaveWorkoutExercises() validates the selection, clones the chosen exercises, replaces the workout's exercises, saves, then reloads the workout for the forms. FinishActiveWorkout() calls AddWorkLog(), which links the log to saved Workout and Exercise records before SaveChanges(). GetWorkoutLogs() feeds the Progress Tracker and IsWorkoutComplete() queries the logs for the selected date.

Database Testing

The database layer was tested manually with the results recorded in Appendix A. The SQLite file was opened in DB Browser for SQLite to check that EF Core had created the expected schema.

Figure A shows PRAGMA table_info('Exercises') executing without errors and returning 10 rows, including the WorkoutId foreign key column and the targeted sets, reps, weight, distance and time columns. This confirms that the exercise subclasses share a single Exercises table. Persistence was then tested across application restarts, including retention of the Progress Tracker history. The NUnit test for SaveWorkoutExercises() covers the service layer save and reload behaviour and passed (1 passed, 0 failed).

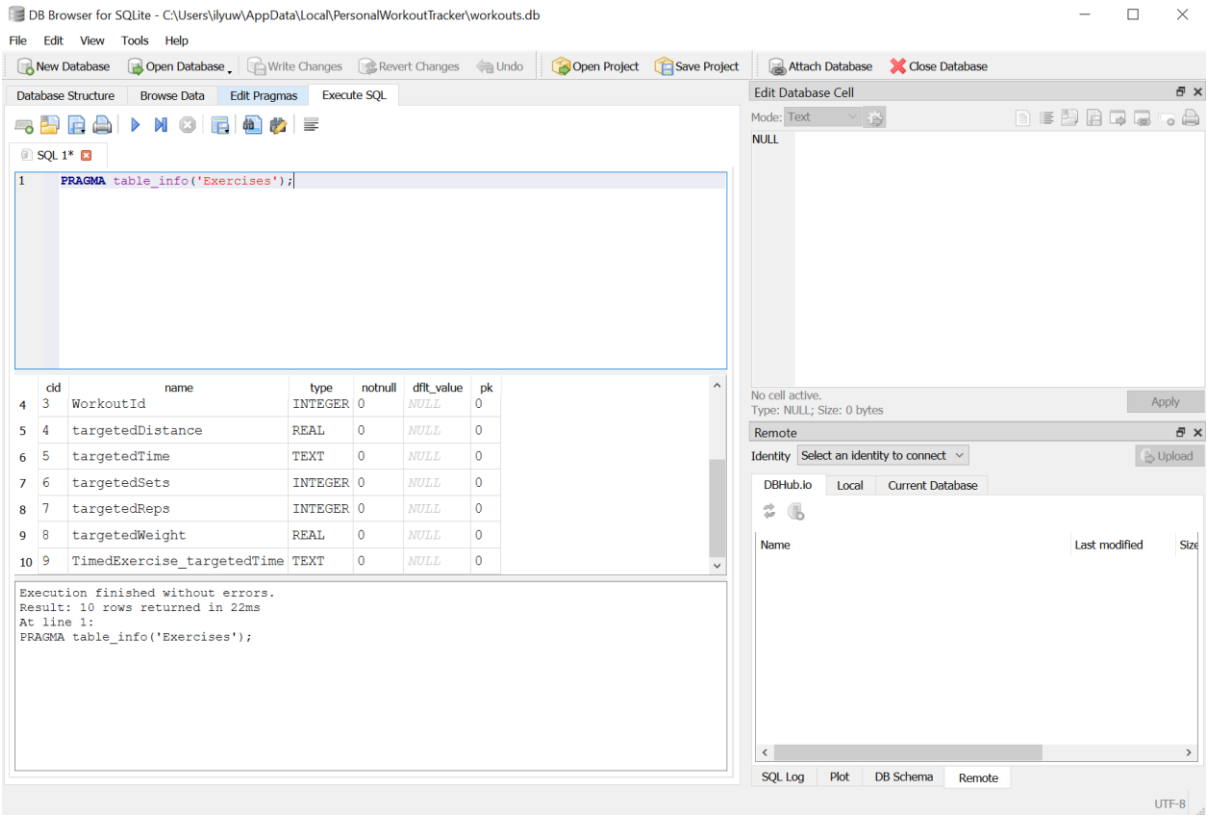


Figure A. Schema check of the Exercises table in the DB Browser for SQLite

Flowchart

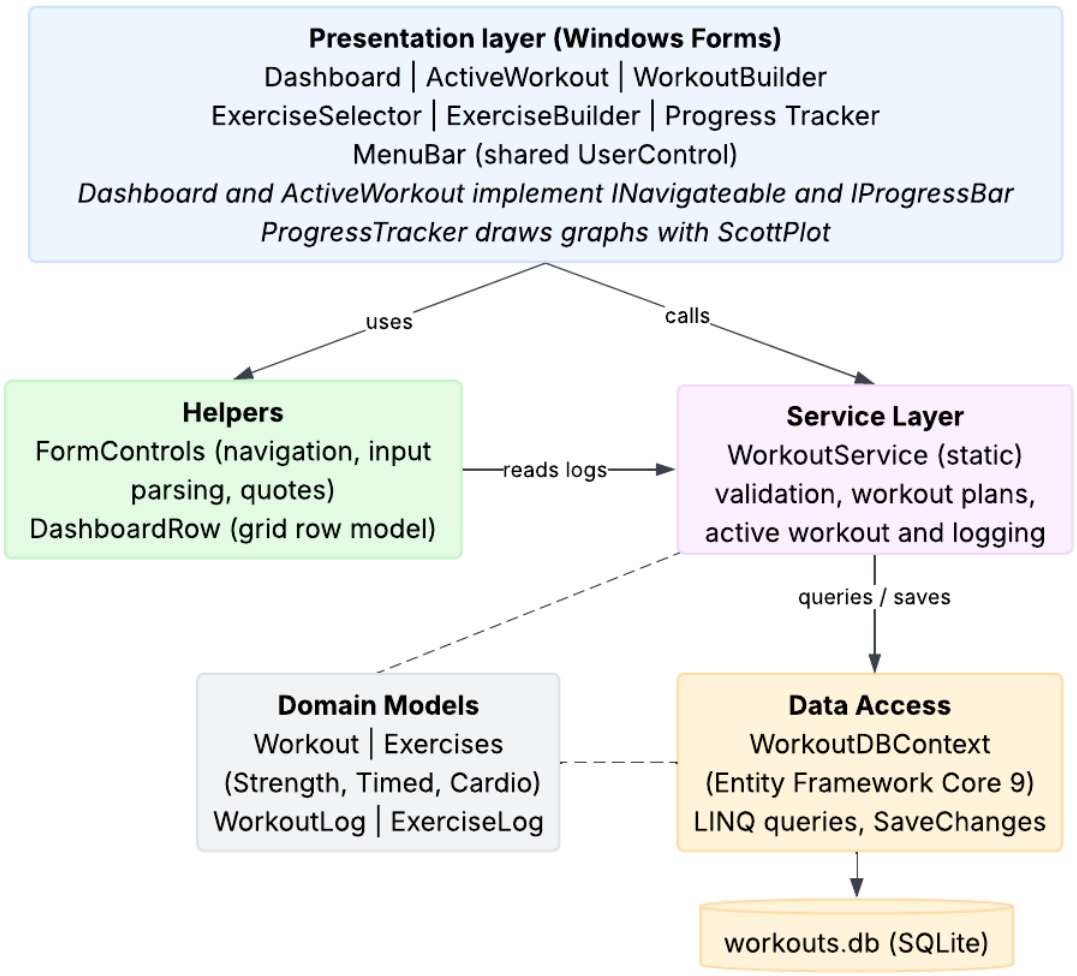


Figure 1: Application architecture

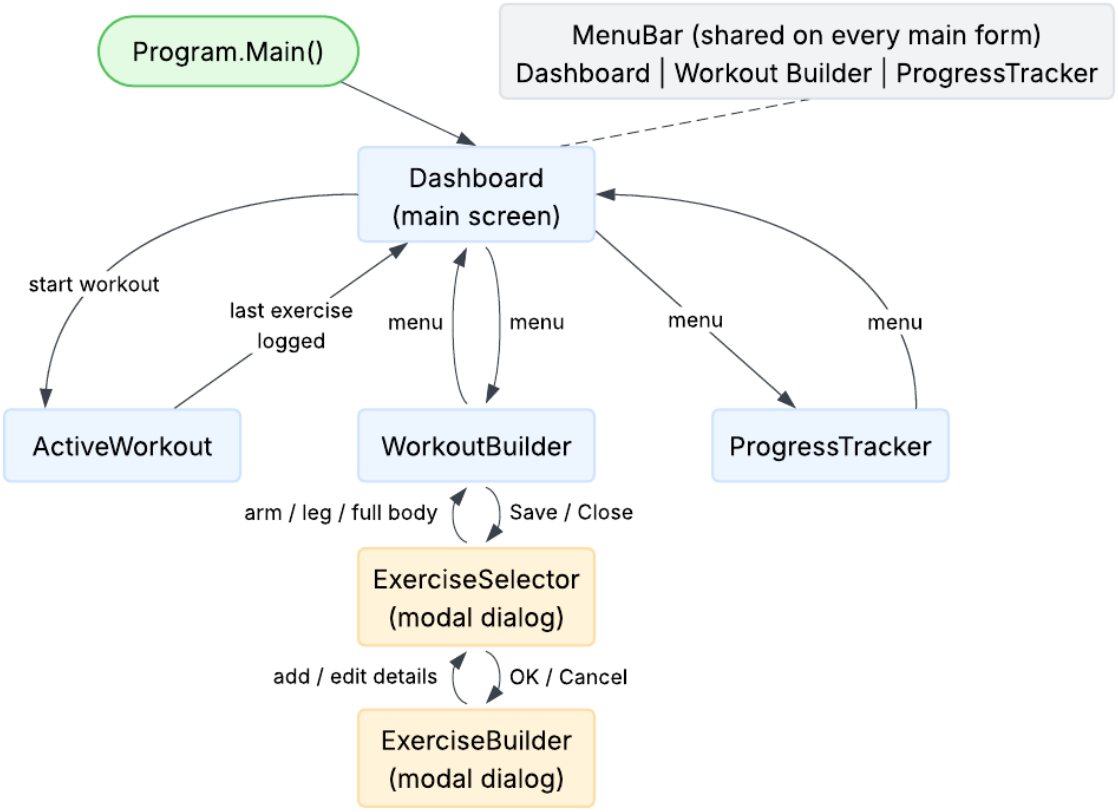


Figure 2: Navigation between forms

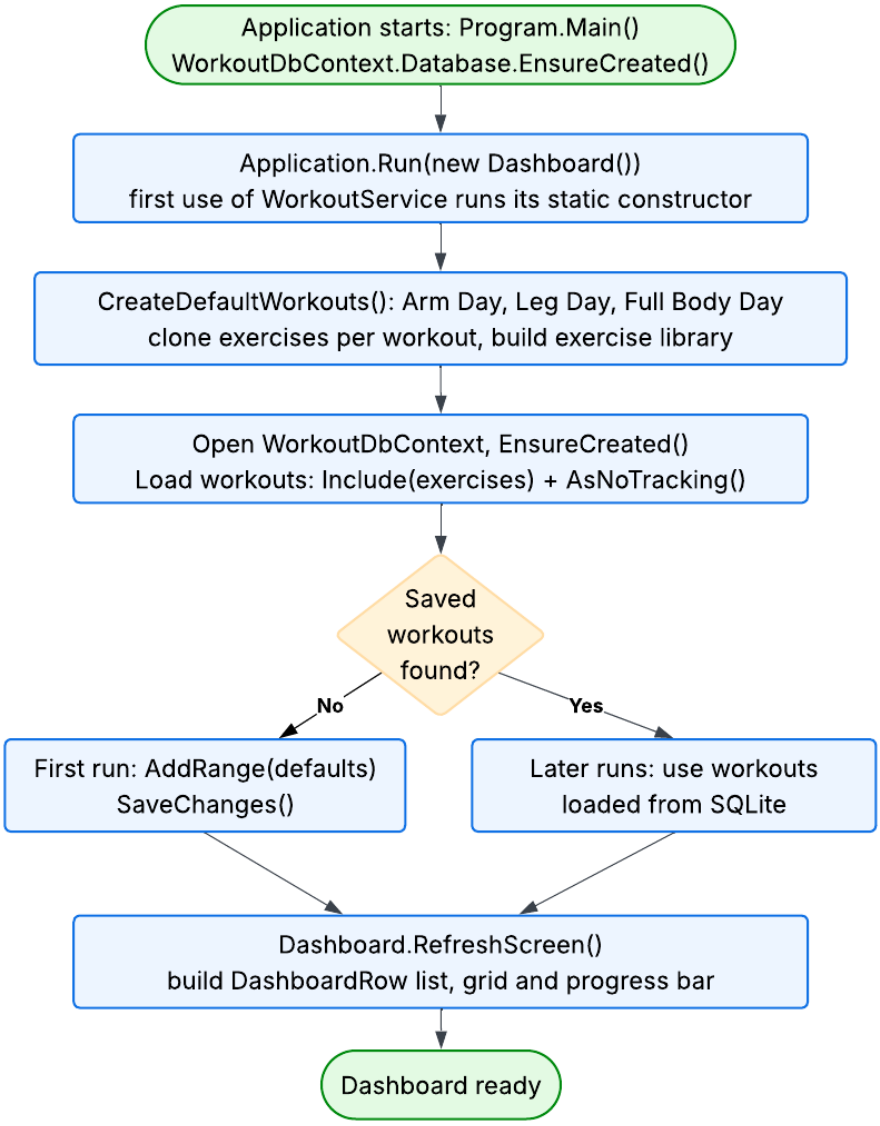


Figure 3: Application startup and database initialisation

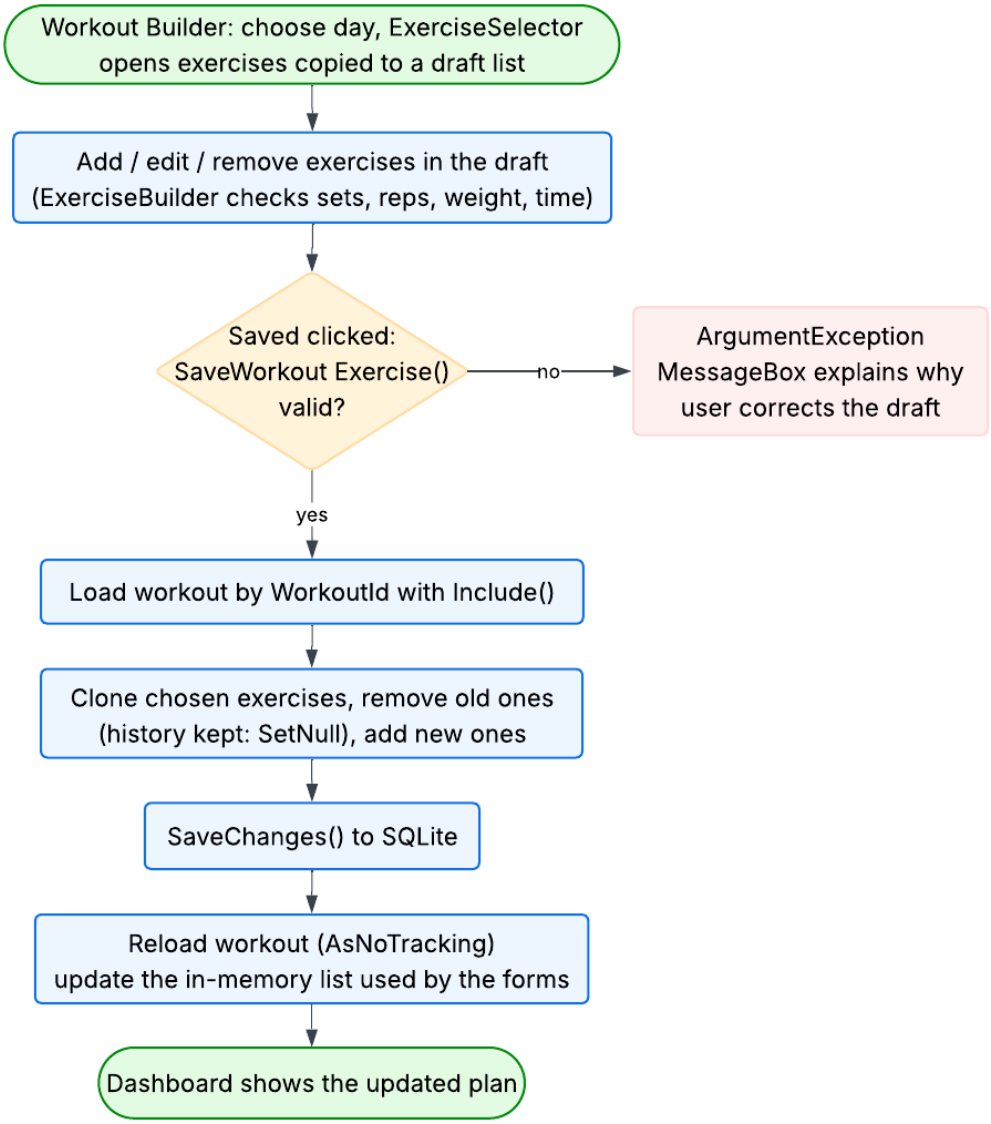


Figure 4: Workout Builder save workflow

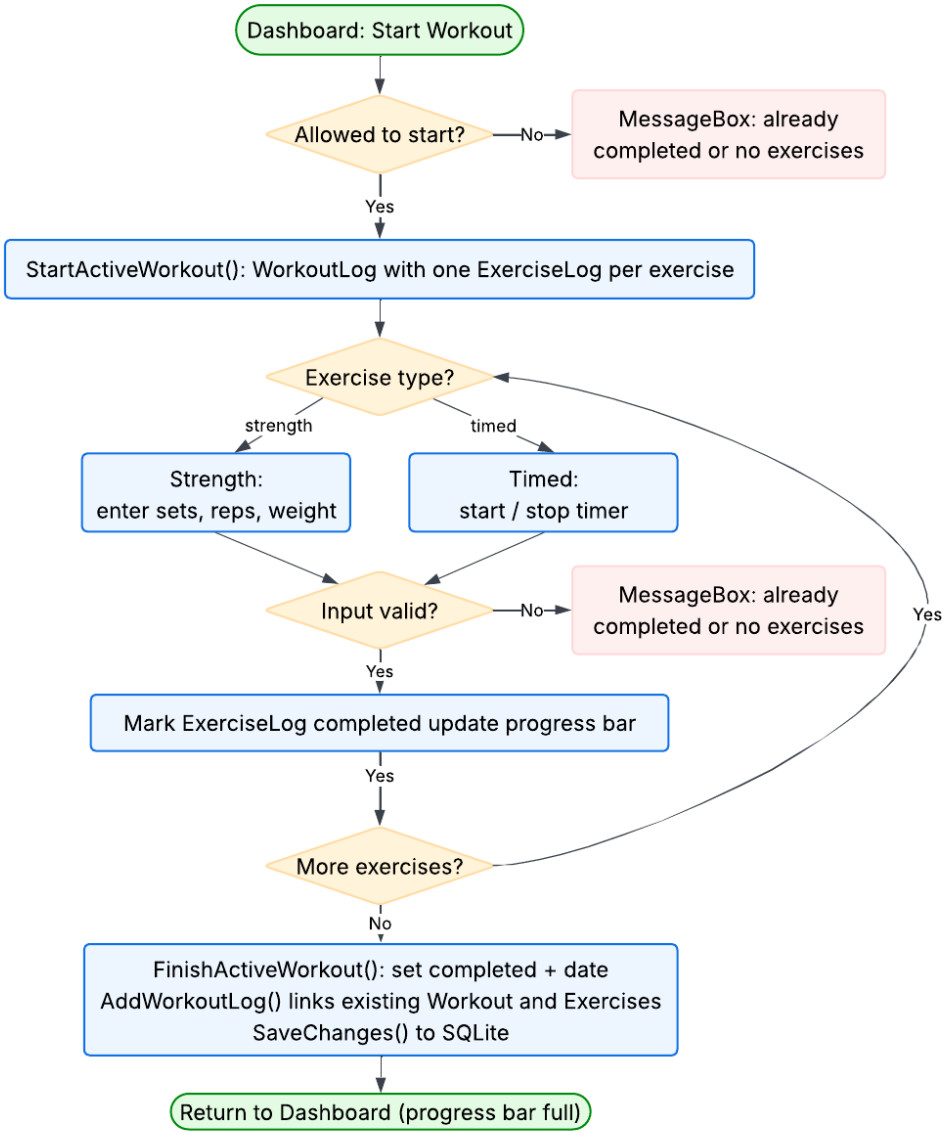


Figure 5: Active workout and logging workflow

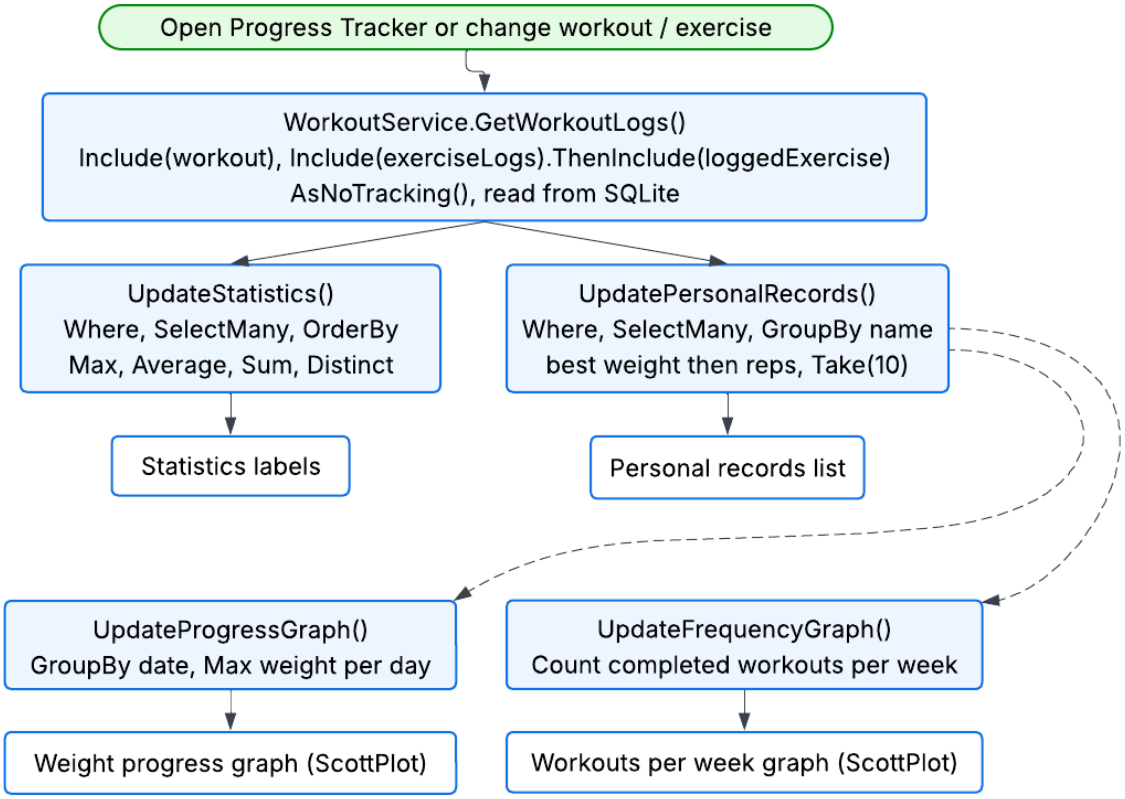


Figure 6: Progress tracker data flow and personal records

Role of team members

This project was split into three distinct roles, these being the lead developer, database and UI designer and feature development and testing. Daniel was responsible for creating the foundational architecture and domain classes. Ily was responsible for planning and creating the UI design as well as creating a persistent database. Vishvak was responsible for developing the workout builder and library functionality and testing the application.

Team Member	Role	Description
Daniel Mercuri	Lead Developer	Started the initial skeleton by first creating the models and defining their inheritances, these being the Workout, Exercise, WorkoutLog and ExerciseLog. Afterwards, started worked on helper methods and services such as FormControls and WorkoutService. Then completed the Dashboard form and the ActiveWorkout form. The Dashboard form involved creating the class DashboardRows to properly display the exercises on the dashboard. Additionally, the ActiveWorkout form involved the creation and use of a timer. Created the interface, INavigatable for handling the carousel and progression in the Dashboard and ActiveWorkout forms. Also created the interface IProgressBar for handling the progress bar functionality in the Dashboard and ActiveWorkoutForm as well.
Ily Ralitza Perez	Database Developer, UI/UX Designer & Developer, Project Documentation	Sketched out the Winforms UI via pen and paper before designing them in Visual Studio using the drag and drop toolbox. Integrated Entity Framework Core 9 with SQLite so workout plans and completed logs persist between launches. In WorkoutDbContext.cs, defined the DbSets, database location and entity relationships, a discriminator for exercise subclasses and SetNull for plan exercises. Updated WorkoutService.cs to load workouts, seed default plans, save and retrieve sessions. Developed and wrote the Progress Tracker LINQ queries and

		methods. Validated the SQLite foreign key configuration and tested persistence across application restarts, including retention of Progress Tracker history. Created the flow charts via Lucid Chart and prepared related documentation for Application and Database Testing.
Vishvak Ananthakrishnan Rameshkumar	Developer and Tester	Developed the Workout Builder functionality, allowing users to select different workout types and add, edit, or remove exercises from their routines. Worked on the Exercise Selector and Exercise Builder forms, where users can customise exercise details such as sets, reps and weight. Also improved the layout to make the forms responsive. Additionally, created an NUnit unit test in UnitTest1.cs to test the SaveWorkoutExercises() method in WorkoutService.cs. The test checks whether exercise details such as name, sets, reps and weight are saved and retrieved correctly. Ran the test in Visual Studio Test Explorer, and it passed successfully with 1 test passed and 0 failed.

Appendix A: Database Design

Database file: workouts.db stored in the Personal Tracker folder under the user's LocalApplicationData directory and created by Database.EnsureCreated().

C:\Users\Daniel\AppData\Local\PersonalWorkoutTracker

Entity	**Mapping**	**Relationships and Behavior**
Workout	DbSet<Workout> Arm Day, Leg Day, or Full Body Day	Has many Exercises (optional, SetNull). Referenced by WorkoutLog (Restrict)
Exercise	DbSet<Exercise> StrengthExercise, TimedExercise and CardioExercise share one Exercises table (ExerciseType discriminator)	Optional WorkoutId foreign key. Referenced by ExerciseLog (Restrict)

WorkoutLog	DbSet<WorkoutLog> A completed workout with status and date.	Refers to a Workout. Owns its ExerciseLogs (cascade)
ExerciseLog	DbSet<ExerciseLog> Recorded sets, reps, weight, time, status and date.	Belongs to a WorkoutLog. Refers to the Exercise it recorded.

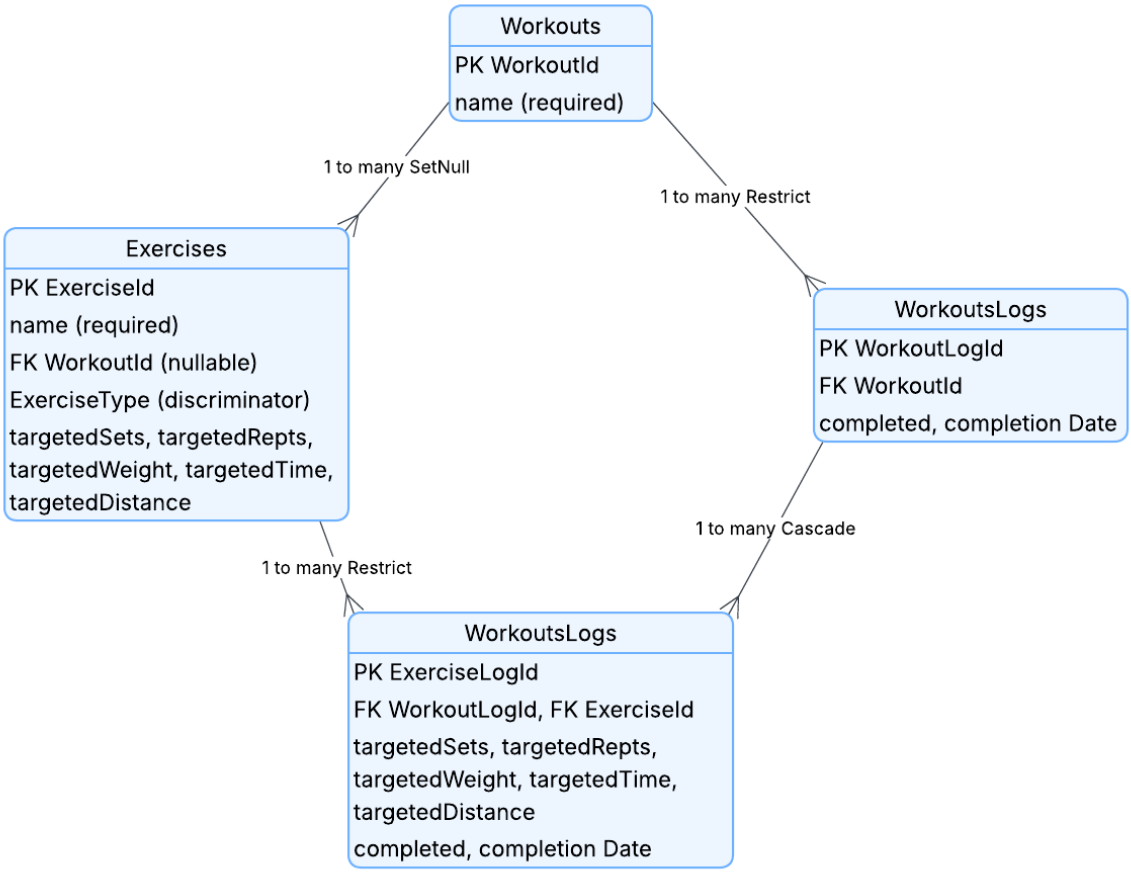


Figure 7: Database schema and relationships

Appendix B: Database Testing Record

Test	**Method**	**Result**
Schema inspection	PRAGMA table_info('Exercises') in DB Browser for SQLite	Executed without errors; 10 rows returned.
Foreign key integrity	PRAGMA foreign_key_check after the foreign key was updated to match DeleteBehavior.SetNull	No violations returned.
Persistence across restarts	Saved plans and completed workouts, then closed and reopened the application.	Saved data and Progress Tracker history retained.

SaveWorkoutExercises() unit	NUnit test in UnitTest1.cs, run in Virtual Studio Test Explorer. It saves and reloads through WorkoutService and the SQLite database, and restores the original exercises afterwards.	1 passed, 0 failed.